

BIG DATA ANALYTICS LAB

BIS701 — VTU 22 Scheme

Programs 2–9 | Short, Exam-Friendly Versions

Course	Big Data Analytics
Code	BIS701
Scheme	VTU 22 Scheme
Programs	2–9 (Program 1 excluded)
Main environments	Jupyter/Spyder, MongoDB, Cloudera, Hive, Hadoop, Spark, HUE

Exam note: These are shortened versions prepared from the supplied lab material. They retain the required operation/concept while removing unnecessary boilerplate. Random MongoDB output will vary.

Contents

Program 2	MapReduce Matrix Multiplication
Program 3	MapReduce Weather Data
Program 4	MapReduce MovieLens Tags
Program 5	MongoDB Count, Sort, Limit, Skip and Aggregate
Program 6	Pig Latin: Sort, Group, Join, Project and Filter
Program 7	Hive Database, Table, View, Function and Index Operations
Program 8	Word Count using Hadoop MapReduce and PySpark
Program 9	CDH and HUE Data Analysis and Reports

Program 2 – MapReduce Matrix Multiplication

Question: Develop a Map Reduce program for Matrix Multiplication. Execute in Jupyter/Spyder.

Environment: Windows — Jupyter Notebook or Spyder. This version is a Python MapReduce simulation and does not require Hadoop.

Code

```
from collections import defaultdict

def mapreduce(A,B):
    d=defaultdict(list)
    for i in range(len(A)):
        for j in range(len(B[0])):
            for k in range(len(B)):
                d[i,j].append(A[i][k]*B[k][j])
    return {key:sum(value) for key,value in d.items()}

A=[[1,2,3],[4,5,6]]
B=[[7,8],[9,10],[11,12]]

R=mapreduce(A,B)

for i in range(len(A)):
    print([R[i,j] for j in range(len(B[0]))])
```

Execution

Run the complete code in Jupyter/Spyder.

Expected / Typical Output

```
[58, 64]
[139, 154]
```

Important: No external package is required; defaultdict is part of Python's standard library.

Program 3 – MapReduce Weather Data

Question: Develop a map reduce program that mines weather data and displays an appropriate message indicating the weather conditions of the day. Execute in Jupyter/Spyder.

Environment: Windows — Jupyter Notebook or Spyder.

Code

```
from collections import Counter

data="""2024-04-01,Sunny
2024-04-01,Rainy
2024-04-01,Sunny
2024-04-02,Cloudy
2024-04-02,Cloudy
2024-04-02,Rainy
2024-04-03,Sunny"""

def mapper(line):
    return tuple(line.split(','))

def reducer(data):
    d={}
    for date,weather in data:
        d.setdefault(date,[]).append(weather)
    for date,w in d.items():
        print(f"Weather on {date}: {Counter(w).most_common(1)[0][0]}")

mapped=[mapper(x) for x in data.splitlines()]
reducer(mapped)
```

Execution

Run the complete code in Jupyter/Spyder.

Expected / Typical Output

```
Weather on 2024-04-01: Sunny
Weather on 2024-04-02: Cloudy
Weather on 2024-04-03: Sunny
```

Important: The code uses the supplied sample weather records directly, so no separate file is required.

Program 4 – MapReduce MovieLens Tags

Question: Develop a Map Reduce program to find the tags associated with each movie by analyzing MovieLens data. Execute in Jupyter/Spyder environment.

Environment: Windows — Jupyter Notebook or Spyder. Keep the two CSV files in the same working folder.

Code

```
import csv

movies={}
with open("movies.csv") as f:
    for r in csv.DictReader(f):
        movies[r["movieId"]]=r["title"]

d={}
with open("tags.csv") as f:
    for r in csv.DictReader(f):
        d.setdefault(r["movieId"],set()).add(r["tag"])

for movie,tags in d.items():
    print(f"Movie: {movies.get(movie,'Unknown Title')}, Tags: {' '.join(sorted(tags))}")
```

Required CSV files

```
movies.csv
movieId,title
1,Toy Story
2,Jumanji
3,Grumpier Old Men

tags.csv
userId,movieId,tag,timestamp
10,1,funny,1234567890
11,1,animation,1234567891
12,2,adventure,1234567892
13,2,game,1234567893
14,1,funny,1234567894
15,3,romantic,1234567895
```

Execution

Run the code in Jupyter/Spyder with movies.csv and tags.csv in the same folder.

Expected / Typical Output

```
Movie: Toy Story, Tags: animation, funny
Movie: Jumanji, Tags: adventure, game
Movie: Grumpier Old Men, Tags: romantic
```

Important: csv is a Python standard-library module. Duplicate tags are removed using a set.

Program 5 – MongoDB Count, Sort, Limit, Skip and Aggregate

Question: Implement functions: Count, Sort, Limit, Skip and Aggregate using MongoDB.

Environment: Windows — MongoDB Shell (mongosh).

Code

```
use experimentFiva

db.createCollection("myCollection")

for(let i=0;i<10000;i++)
  db.myCollection.insertOne({
    name:"Person"+i,
    age:Math.floor(Math.random()*100),
    city:["New York","Los Angeles","Chicago","Houston","Phoenix"][Math.floor(Math.random()*5)]
  })

db.myCollection.countDocuments()

db.myCollection.find().sort({age:1}).limit(5)

db.myCollection.find().skip(9999)

db.myCollection.find().limit(2)

db.myCollection.find({age:{>50}})

db.myCollection.aggregate([
  {$group: {_id: "$city", count: {$sum:1}}},
  {$sort: {count:-1}}
])
```

Execution

Open mongosh and paste/run the code.

Expected / Typical Output

Count: 10000. The remaining outputs are documents/results and vary because age and city are generated randomly.

Important: The original question's sample step using age > 99 normally returns no documents because generated ages are 0–99. The study version uses age > 50 so the filter demonstrates the operation meaningfully.

Program 6 – Pig Latin: Sort, Group, Join, Project and Filter

Question: Write Pig Latin scripts to sort, group, join, project and filter the data.

Environment: Cloudera / Pig in MapReduce mode with HDFS.

Code

```
employees = LOAD '/user/cloudera/employees.txt' USING PigStorage(',')
AS (id:int,name:chararray,age:int,dept:int,salary:int);

departments = LOAD '/user/cloudera/departments.txt' USING PigStorage(',')
AS (dept:int,dept_name:chararray);

filtered = FILTER employees BY age > 28;

projected = FOREACH filtered GENERATE id,name,salary;

sorted = ORDER projected BY salary DESC;

grouped = GROUP employees BY dept;

joined = JOIN employees BY dept, departments BY dept;

DUMP sorted;
DUMP grouped;
DUMP joined;
```

Compile / Run / HDFS Commands

```
hdfs dfs -mkdir -p /user/cloudera
hdfs dfs -put employees.txt /user/cloudera/
hdfs dfs -put departments.txt /user/cloudera/

pig -x mapreduce employee_analysis.pig
```

Required input files

```
employees.txt
101,John,30,1,50000
102,Sam,28,2,60000
103,Anna,32,1,75000
104,David,29,3,62000
105,Lily,27,2,58000

departments.txt
1,HR
2,Finance
3,IT
```

Important: The shortened script uses DUMP to display the three results directly, avoiding unnecessary STORE commands for an exam demonstration.

Program 7 – Hive Database, Table, View, Function and Index Operations

Question: Demonstrate Hive commands for creating and altering databases and tables, creating and altering views, adding and dropping functions, and creating, rebuilding and dropping an index.

Environment: Cloudera Hive shell / HUE Hive Query Editor.

Code

```
CREATE DATABASE employee_db;
USE employee_db;

ALTER DATABASE employee_db SET DBPROPERTIES ('Owner'='Admin');

CREATE TABLE employees(
    emp_id INT,
    name STRING,
    age INT,
    dept_id INT,
    salary FLOAT
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

LOAD DATA INPATH '/user/cloudera/employees.txt' INTO TABLE employees;

ALTER TABLE employees ADD COLUMNS (email STRING);

ALTER TABLE employees RENAME TO employees_new;
ALTER TABLE employees_new RENAME TO employees;

CREATE VIEW high_salary_employees AS
SELECT emp_id,name,salary FROM employees WHERE salary>50000;

ALTER VIEW high_salary_employees AS
SELECT emp_id,name,age,salary FROM employees WHERE salary>60000;

DROP VIEW high_salary_employees;

ADD JAR /user/cloudera/custom_udf.jar;
CREATE FUNCTION to_upper AS 'com.example.hiveudf.ToUpperUDF';
SELECT to_upper(name) FROM employees;
DROP FUNCTION to_upper;

CREATE INDEX emp_dept_idx ON TABLE employees(dept_id)
AS 'org.apache.hadoop.hive.ql.index.compact.CompactIndexHandler'
WITH DEFERRED REBUILD;

ALTER INDEX emp_dept_idx ON employees REBUILD;
DROP INDEX emp_dept_idx ON employees;

DESCRIBE employees;
SELECT * FROM employees LIMIT 5;
SHOW TABLES;

DROP TABLE employees;
DROP DATABASE employee_db CASCADE;
```

Sample Input

```
employees.txt
101,John,30,1,50000
102,Sam,28,2,60000
103,Anna,32,1,75000
104,David,29,3,62000
105,Lily,27,2,58000
```

Execution

Execute in the Cloudera Hive shell or HUE Hive Query Editor. Ensure the UDF JAR exists at the specified HDFS path before running.

Important: The original material's final database drop is placed at the end so that it does not remove the database before the table operations are complete. Hive index support depends on the Hive version/configuration; if the lab image does not support indexes, follow the lecturer's exact index syntax.

Program 8 – Word Count using Hadoop MapReduce and PySpark

Question: Implement a Word Count program in Hadoop and Spark.

Environment: Cloudera / Hadoop MapReduce and PySpark. Input is stored in HDFS.

Hadoop MapReduce — WordCount.java

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {
    public static class Map extends Mapper<Object,Text,Text,IntWritable> {
        IntWritable one=new IntWritable(1);
        Text word=new Text();

        public void map(Object k,Text v,Context c) throws IOException,InterruptedException {
            StringTokenizer s=new StringTokenizer(v.toString());
            while(s.hasMoreTokens()) {
                word.set(s.nextToken());
                c.write(word,one);
            }
        }
    }

    public static class Reduce extends Reducer<Text,IntWritable,Text,IntWritable> {
        public void reduce(Text k,Iterable<IntWritable> v,Context c)
            throws IOException,InterruptedException {
            int sum=0;
            for(IntWritable x:v) sum+=x.get();
            c.write(k,new IntWritable(sum));
        }
    }

    public static void main(String[] a) throws Exception {
        Job j=Job.getInstance(new Configuration(),"WordCount");
        j.setJarByClass(WordCount.class);
        j.setMapperClass(Map.class);
        j.setCombinerClass(Reduce.class);
        j.setReducerClass(Reduce.class);
        j.setOutputKeyClass(Text.class);
        j.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(j,new Path(a[0]));
        FileOutputFormat.setOutputPath(j,new Path(a[1]));
        System.exit(j.waitForCompletion(true)?0:1);
    }
}
```

PySpark — wordcount.py

```
from pyspark.sql import SparkSession

spark=SparkSession.builder.appName("WordCount").getOrCreate()

f=spark.sparkContext.textFile("hdfs://localhost:9000/user/cloudera/input/sample.txt")

wc=f.flatMap(lambda x:x.split()).map(lambda x:(x,1)).reduceByKey(lambda a,b:a+b)

for w,c in wc.collect():
    print(w,c)

wc.saveAsTextFile("hdfs://localhost:9000/user/cloudera/output_spark")
spark.stop()
```

Compile / Run / HDFS Commands

```
# Hadoop
javac -classpath `hadoop classpath` -d . WordCount.java
jar cf wc.jar WordCount*.class
hdfs dfs -mkdir -p /user/cloudera/input
hdfs dfs -put sample.txt /user/cloudera/input/
hadoop jar wc.jar WordCount /user/cloudera/input /user/cloudera/output
hdfs dfs -cat /user/cloudera/output/part-r-000000

# Spark
spark-submit wordcount.py
hdfs dfs -cat /user/cloudera/output_spark/part-000000
```

Sample Input

Hello Hadoop Hello Spark Hello World Spark is fast Hadoop is slow

Important: The original PySpark source contained the typo 'qspark. sparkContext'; the corrected form is spark.sparkContext. The input file must be uploaded to HDFS before both jobs.

Program 9 – CDH and HUE Data Analysis and Reports

Question: Use CDH (Cloudera Distribution for Hadoop) and HUE (Hadoop User Interface) to analyze data and generate reports for sample data sets.

Environment: Cloudera CDH + HUE + HDFS + Hive.

Code

```
sudo service cloudera-scm-server start
sudo service cloudera-scm-agent start

hdfs dfs -mkdir -p /user/cloudera/data
hdfs dfs -put employees.csv /user/cloudera/data/
```

Hive SQL

```
CREATE DATABASE IF NOT EXISTS company;
USE company;

CREATE TABLE employees(
    id INT,
    name STRING,
    department STRING,
    salary INT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

LOAD DATA INPATH '/user/cloudera/data/employees.csv' INTO TABLE employees;

SELECT * FROM employees;

SELECT department,MAX(salary) AS highest_salary
FROM employees
GROUP BY department;

SELECT name,department,salary
FROM employees
WHERE salary>65000;

SELECT department,COUNT(*) AS employee_count
FROM employees
GROUP BY department;
```

Sample Input

```
id,name,department,salary
1,John,IT,70000
2,Alice,HR,60000
3,Bob,IT,75000
4,Charlie,Finance,80000
5,David,HR,62000
6,Eva,IT,72000
7,Frank,Finance,81000
8,Grace,HR,65000
```

HUE Instructions

Open <http://localhost:8888> in a browser, log in with the lab HUE credentials, open Query Editors → Hive, and run the SQL. Results can be exported from HUE; a chart/dashboard can be created from the employee-count query.

Important: The HDFS commands can be run in the terminal, while the Hive SQL can be run in HUE's Hive Query Editor.